

Roadmap del Sistema de URIs — SOUP v2

Objetivo: implementar un sistema de URIs estable, legible y universal para **bandas** y **álbumes** (extensible a personas, tours, sellos, negocios y productos), con mínima fricción sobre el backend actual.

Índice

- [1\) Prompts para agente de IA](#)
- [2\) Modelos estándar de código \(plantillas\)](#)
- [3\) Reglas de implementación \(invariantes\)](#)
- [4\) Esquema de arquitectura \(alto nivel\)](#)
- [5\) Checks y tests](#)
- [6\) Recomendaciones](#)
- [7\) Tutorial para el humano \(paso a paso\)](#)

1) Prompts para agente de IA

A. Crear tabla y modelo de registro de URIs

Objetivo: agregar tabla `entity_uri_registry` y su modelo SQLAlchemy; exponer `uri` en respuestas públicas.

Prompt (copiar/pegar):

```
En `backend/app`, crea el modelo `EntityUriRegistry(gid, tipo, slug_actual,
slugs_historicos, owner_table, owner_id, created_at, updated_at)`.
Genera migración SQL/DDL.
Agrega un helper `slugify(name)` y `short_from_gid(gid)`.
Inserta hooks en `POST /businesses` y `POST /products` para crear `gid`
(UUIDv7/ULID), generar `slug` y guardar en el registry.
Devuelve el campo `uri` en `GET /public/businesses` y `GET /public/products`.
```

B. Resolver y subrecursos

Objetivo: servir URLs públicas y canónicas, más subrecursos universales.

Prompt:

```
Crea router `public_uri_router`:
- `GET /public/{tipo}/{slug_or_gid}` → 200 (payload) o 301 al canónico
`https://id.soup.market/{tipo}/{gid}`.
```

```
- Subrutas: `/about`, `/members`, `/discography`, `/tours`, `/media`, `/links` (devuelven listas vacías/null por defecto).  
Incluye negociación `?lang=&format=`.
```

C. Frontend y estado vacío amable

Objetivo: UI/UX con tabs y estados vacíos.

Prompt:

```
En pantallas públicas, agrega tabs: About, Members, Discography, Media, Links.  
Cada tab debe mostrar un estado vacío con CTA "Agregar" o "Importar" (Bandcamp/YouTube).  
No rompas listados existentes: sólo agrega el campo `uri` en tarjetas y vistas.
```

D. Tests de creación, resolución y 301

Objetivo: cobertura básica del flujo.

Prompt:

```
Agrega tests (pytest) para:  
- creación de negocio/producto crea `gid+slug`;  
- `GET /public/{tipo}/{slug}` resuelve a `gid`;  
- `GET /public/products` y `/public/businesses` incluyen `uri`;  
- renombre genera 301 desde slugs viejos al slug actual.
```

2) Modelos estándar de código (plantillas)

A. Modelo SQLAlchemy (nuevo)

```
# backend/app/models_uri.py  
from sqlalchemy import Column, String, DateTime, ARRAY, func  
from sqlalchemy.dialects.postgresql import UUID  
from sqlalchemy.orm import Mapped, mapped_column  
from uuid import uuid4  
from app.database import Base  
  
class EntityUriRegistry(Base):  
    __tablename__ = "entity_uri_registry"
```

```

    gid: Mapped[str] = mapped_column(String, primary_key=True)
# ULID/UUIDv7 string
    tipo: Mapped[str] = mapped_column(String, index=True)          # band|album|
person|tour|label|business|product
    slug_actual: Mapped[str] = mapped_column(String, index=True,
unique=False)
    slugs_historicos: Mapped[list[str] | None] = mapped_column(ARRAY(String),
nullable=True)
    owner_table: Mapped[str] = mapped_column(String)              #
"negocios"|"productos"|...
    owner_id: Mapped[str] = mapped_column(UUID(as_uuid=True), index=True)
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    updated_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now(), onupdate=func.now())

```

DDL (si necesitas SQL directo):

```

CREATE TABLE IF NOT EXISTS entity_uri_registry (
    gid TEXT PRIMARY KEY,
    tipo TEXT NOT NULL,
    slug_actual TEXT NOT NULL,
    slugs_historicos TEXT[] NULL,
    owner_table TEXT NOT NULL,
    owner_id UUID NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_entity_uri_tipo ON entity_uri_registry (tipo);
CREATE INDEX IF NOT EXISTS ix_entity_uri_slug ON entity_uri_registry
(slug_actual);

```

B. Utilidades (nuevo)

```

# backend/app/utils/uri.py
import re, unicodedata, hashlib

def slugify(s: str) -> str:
    s = unicodedata.normalize("NFKD", s).encode("ascii",
"ignore").decode("ascii")
    s = re.sub(r"[^a-zA-Z0-9]+", "-", s).strip("-").lower()
    return re.sub(r"-{2,}", "-", s)

def short_from_gid(gid: str, length=8) -> str:
    h = hashlib.blake2b(gid.encode(), digest_size=8).hexdigest()
    return h[:length]

```

C. Esquemas Pydantic (nuevo)

```
# backend/app/schemas_uri.py
from pydantic import BaseModel
from typing import Optional, List

class UriResponse(BaseModel):
    uri: str
    tipo: str
    gid: str
    slug: str
    links: Optional[List[dict]] = None
    class Config:
        from_attributes = True
```

D. Router (stubs)

```
# backend/app/routers/public_uri_router.py
from fastapi import APIRouter, HTTPException
router = APIRouter(prefix="/public", tags=["Public URIs"])

@router.get("/{tipo}/{slug_or_gid}")
def resolve(tipo: str, slug_or_gid: str):
    # Buscar por gid exacto o por slug_actual; si hay slug histórico → 301
    ...

@router.get("/{tipo}/{slug_or_gid}/discography")
def discography(tipo: str, slug_or_gid: str):
    # Si tipo=band → productos por business_id
    ...
```

3) Reglas de implementación (invariantes)

1. Mínimos obligatorios

2. Band (negocio): , .

3. Álbum (producto): , .

4. El resto **opcional** por defecto (bio, media, links, integrantes, tours, etc.).

5. GID

6. ULID/UUIDv7 generado al crear la entidad; **nunca** cambia.

7. Slug

8. Derivado de `nombre` con `slugify`.

9. Si cambia el nombre, agregar el viejo a `slugs_historicos` y servir **301** desde slugs viejos al actual.

10. URL pública amigable: `https://soup.market/{tipo}/{slug}~{short}`.

11. URL canónica técnica: `https://id.soup.market/{tipo}/{gid}`.

12. Subrecursos universales

13. `/about`, `/members`, `/discography`, `/tours`, `/media`, `/links`.

14. Responden vacío hasta que haya contenido.

15. Marketplace

16. Los álbumes (productos) aparecen en **ambos** lugares: dentro del URI de la banda (`.../discography`) y en el marketplace general `/public/products`.

17. Compatibilidad

18. No romper contratos existentes; sólo **agregar** `uri` a respuestas públicas.

4) Esquema de arquitectura (alto nivel)

[FastAPI Routers]

- └ `/businesses` (crear banda = negocio)
- └ `/products` (crear álbum = producto)
- └ `/public` (listados existentes)
 - └ `/public/{tipo}/{slug|gid}` ← NUEVO resolver + subrecursos

[DB]

- └ `negocios` (band)
- └ `productos` (album)
- └ `entity_uri_registry` ← NUEVA

[Flujo creación]

POST `/businesses` -> crea negocio -> `gid+slug` -> `registry` -> responde con `uri`

POST `/products` -> crea producto -> `gid+slug` -> `registry` -> responde con `uri`

Listados `/public/*` incluyen `uri` (back-compatible)

5) Checks y tests

Unit

- `slugify()` normaliza acentos, espacios y símbolos.
- `short_from_gid()` es determinista.

Integración (pytest)

- Crear negocio → inserta en `entity_uri_registry` y responde con `uri`.
- Crear producto → idem.
- `GET /public/{tipo}/{slug}` resuelve a `gid`.
- Renombrar banda → antiguo slug responde **301** con `Location` al slug actual.
- `GET /public/products` y `GET /public/businesses` incluyen `uri`.

End-to-end (manual/Playwright)

- En UI pública, tabs de subrecursos muestran estados vacíos y CTAs.

No-regresión

- CRUD de productos y cálculos existentes (precios, COGS, etc.) siguen iguales.
-

6) Recomendaciones

- **IA & workflow:**
 - **Continue (VS Code)** para iterar rápido dentro de archivos.
 - **aider (CLI)** para cambios multi-archivo con commits atómicos.
 - **Gemini Code Assist / Windsurf** para transformaciones grandes coordinadas.
 - **Git:** rama `feature/uri-system`; PR pequeños y encadenados:
 - **tabla+modelo**, 2) hooks de creación, 3) resolver, 4) subrecursos vacíos, 5) tests.
 - **Logs:** logs de debug en resolver y hooks.
 - **Datos:** mantener slugs viejos (no borrar); 301 para retrocompatibilidad.
 - **i18n:** `?lang=` y versiones multilenguaje en `about` a futuro.
-

7) Tutorial para el humano (paso a paso)

0. Backup rápido

- Exportá la base (`pg_dump`) y confirmá que podés restaurarla.

1. Crear tabla y modelo

- Agregá `models_uri.py` con la clase `EntityUriRegistry`.
- Ejecutá migración/DDL.
- Importá el modelo en `main.py` (o donde se inicializan modelos) antes de `create_all`.

2. Utilidades

- Crea `app/utils/uri.py` con `slugify` y `short_from_gid`.

3. Hooks de creación

- En los handlers de **crear negocio** y **crear producto**, tras persistir:
- Genera `gid` (UUIDv7/ULID).
- `slug = slugify(nombre) + "~" + short_from_gid(gid)`.
- Inserta en `entity_uri_registry`.
- Adjunta `uri` en la respuesta del recurso.
- En los listados públicos, agrega `uri` al payload.

4. Resolver público

- Crea `routers/public_uri_router.py` con:
- `GET /public/{tipo}/{slug_or_gid}` (resolve/301)
- subrecursos `/about`, `/members`, `/discography`, `/tours`, `/media`, `/links` (vacíos al inicio)
- Montalo en `main.py` junto con el resto de routers.

5. Discografía

- En `.../discography`, si `tipo=band`, buscar productos por `business_id` (álbumes de la banda).

6. Probar (quick & dirty)

```
# Crear banda
curl -X POST http://localhost:8000/businesses/ \
-H 'Authorization: Bearer TOKEN' \
-H 'Content-Type: application/json' \
-d '{"nombre": "Banda Ñam", "propietario_id": "..."}'

# Crear álbum
curl -X POST http://localhost:8000/products/ \
-H 'Authorization: Bearer TOKEN' \
-H 'Content-Type: application/json' \
-d '{"nombre": "Salsa Verde 2025", "business_id": "..."}'

# Listados (deben incluir "uri")
curl http://localhost:8000/public/businesses
curl http://localhost:8000/public/products

# Resolver
curl -i http://localhost:8000/public/band/banda-nam~x7k3
curl -i http://localhost:8000/public/album/salsa-verde-2025~p4f9
```

7. Tests

- Crea `tests/test_uri_resolver.py` con los casos de la sección 5.

8. UI mínima

- Agrega tabs y estados vacíos en páginas públicas: About, Members, Discography, Media, Links.
 - Añade el campo `uri` en tarjetas/listados sin romper estilos ni contratos.
-

Lista de tareas (checklist)

-

Nota: Todos los campos enriquecidos (bio, integrantes, tours, media, links) son **opcionales** y comienzan vacíos. El sistema de URIs debe funcionar desde el día 1 únicamente con `nombre` y relaciones mínimas (banda ↔ álbum).